



PlaySBC

Product Guide

Features, Architecture, and Administration

Version 2.6.0 | Final public MIT baseline

Contributor

Sudheer Kumar Vatrapi

August 2026

Document Control

Field	Value
Product	PlaySBC
Release	v2.6.0
Status	Final public MIT engineering baseline
Contributor	Sudheer Kumar Vatrapi
Audience	SIP engineers, platform engineers, lab administrators, and evaluators
Scope	Features, architecture, deployment, administration, operations, evidence, and limitations

Important Product Boundary

PlaySBC v2.6.0 is an engineering and interoperability lab. It is not a production-certified carrier SBC, and this guide does not make unmeasured capacity, compliance, or availability claims.

Contents

- Product overview and feature baseline
- System architecture and protocol behavior
- Docker, kind, minikube, Azure AKS, and real-device deployment models
- Helm lifecycle, operations, observability, and evidence
- Security, HA, recovery, troubleshooting, and honest capacity limits

Product Overview

PlaySBC combines SIP signalling, B2BUA routing, RTPengine media control, AI voice experiments, observability, and evidence-driven regression in one lab platform.

Core Capabilities

- SIP REGISTER and B2BUA calls over UDP, TCP, and TLS.
- Digest registration for synthetic endpoints and real devices.
- RTPengine anchoring with RTP/RTCP, G.711 transcoding, SRTP interworking, and NAT learning.
- Active-active and failure-injection lab foundations with Helm-managed Kubernetes topology.
- Rasa, STT, TTS, DTMF, and scripted AI Voice Gateway regression foundations.
- RFC 5359 call hold/resume signalling profiles.
- Prometheus metrics, Grafana dashboards, canonical ladders, logs, and combined PCAP evidence.

Validated Endpoint Baseline

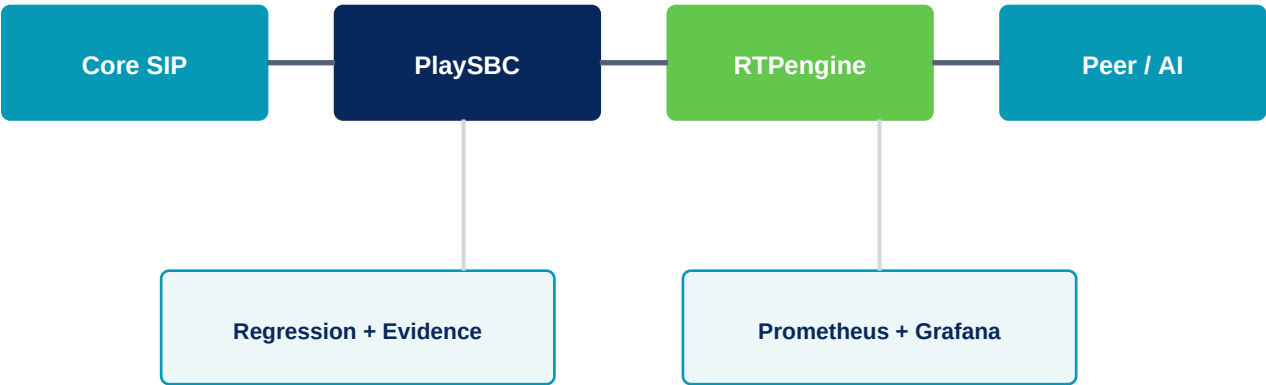
OBI1022 user 1001 and Zoiper user 1002 have been registered through Azure AKS and used for two-way RTPengine-anchored audio. The local real-device lane advertises the Mac LAN address and remains isolated from the full regression cluster.

Public Release Gate

v2.6.0 contains 70 selectable Kubernetes regression profiles, live X/70 launcher progress, macOS host-sleep inhibition, and role-aware merged-PCAP certification.

System Architecture

PlaySBC separates signalling, media anchoring, test orchestration, and observability so each layer can be inspected independently.



Component Responsibilities

Component	Responsibility
PlaySBC	SIP listener, registrar, B2BUA routing, dialog handling, policy, health, and metrics.
RTPengine	Media anchoring, address/port rewriting, G.711 handling, RTP/RTCP, SRTP, and NAT learning.
SIPp agents	Core and peer endpoint roles used by deterministic regression profiles.
Regression runner	Helm profile application, orchestration, verdicts, collection, and HTML/archive output.
Prometheus/Grafana	Metrics retention, dashboards, traffic, media, AI, and HA visibility.

Call Model

The inbound SIP dialog and outbound SIP dialog are separate B2BUA legs with distinct Call-IDs. v2.6.0 packet evidence checks for both roles and both INVITE legs in plain-SIP bridged profiles.

Protocol and Media Behavior

Signalling Baseline

- SIP UDP 5062, SIP TCP 5062, and SIP TLS 5061.
- REGISTER challenge and authenticated registration for configured users.
- INVITE routing to registered contacts, provisional/final responses, ACK, BYE, and CANCEL handling.
- OPTIONS health checks and transport-policy profiles.
- In-dialog hold/resume re-INVITE signalling in the public lab baseline.

Media Baseline

- RTPengine NG control on UDP 2223.
- Local and AKS real-device RTP range 30000-30049/UDP.
- Load-only regression profiles may use a larger profile-scoped RTP range.
- PCMU/PCMA G.711 operation, transcoding evidence, RTP/RTCP direction verdicts, and SRTP interworking profiles.

Evidence Versus Certification

A passing profile proves the behavior and evidence asserted by that profile. It does not automatically certify every RFC clause, endpoint, network, scale, or failure mode.

Deployment Models

Choose one lane for one purpose. Do not reuse cloud values in local clusters or local real-device values in AKS.

Model	Topology	Primary use	Exposure
Docker	Process/container lab	Fast development checks	Local only
kind-playsbc	2 PlaySBC + 2 RTPengine	Canonical 70-profile regression	Docker Desktop
Minikube	Compatibility topology	Kubernetes portability	Local
kind real device	1 PlaySBC + 1 RTPengine	LAN OBi/Zoiper calls	Mac LAN ports
Azure AKS	1+1 readiness baseline	Azure LB, ACR, public SIP/RTP	Cloud resources

Isolation Rules

- Use context kind-playsbc for canonical local regression.
- Use context kind-playsbc-real-device for LAN phone testing.
- Use the AKS context only with Azure values and LoadBalancer resources.
- Verify context, image tags, advertised addresses, and RTP range before a run.

Docker Administration

Use Docker for fast local protocol and application checks before Kubernetes. Docker Desktop must be running on macOS.

```
cd /path/to/PlaySBC
docker info

PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \
python3 tools/run_regression_suite.py \
--skip-sipp-smoke \
--all-b2bua-profiles \
--timeout 420
```

Output: logs/reports/latest.html

Administration Notes

- Confirm Docker has sufficient CPU, memory, disk, and battery/power before media or load work.
- Use versioned images; do not assume latest points at the release gate.
- Inspect container logs and generated HTML before escalating a failed scenario.

kind and Minikube Administration

kind is the canonical full regression environment. kind nodes are Docker containers, so Docker Desktop must remain running. Minikube is a compatibility lane.

```
open -a Docker
until docker info >/dev/null 2>&1; do sleep 5; done

kind get clusters
kubectl config use-context kind-playsbc
kubectl config set-context --current --namespace=playsbc
kubectl get nodes
kubectl get pods -n playsbc
```

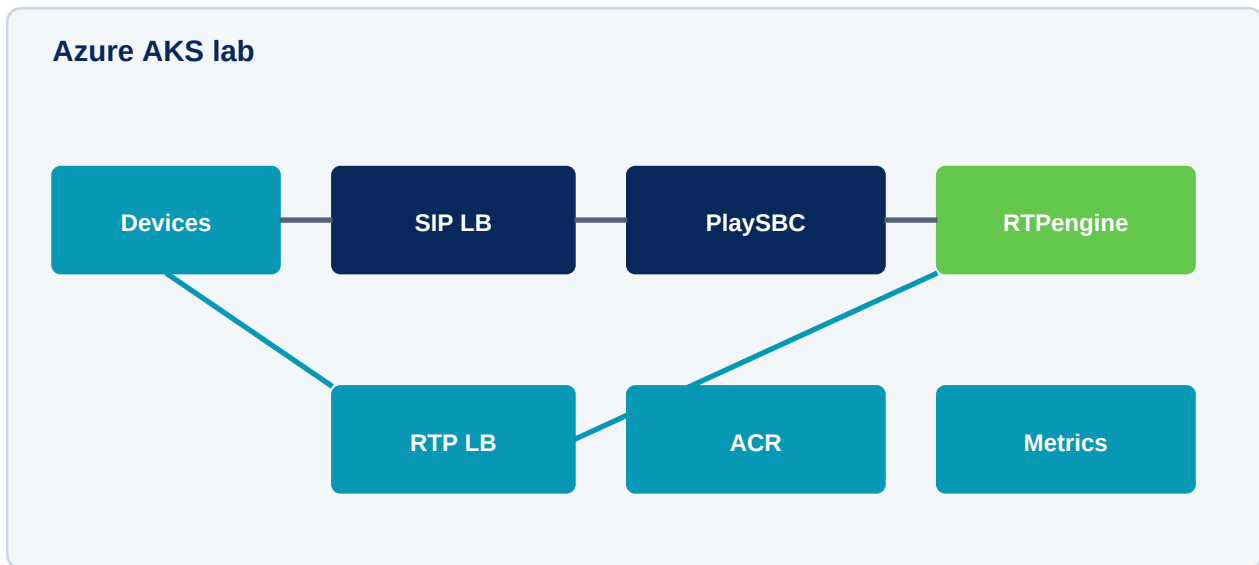
Upgrade v2.6.0

```
export PLAYSBC_VERSION=2.6.0

helm upgrade --install playsbc \
https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v${PLAYSBC_VERSION}/playsbc-${PLAYSBC_VERSION}.tgz \
-n playsbc --create-namespace --atomic --wait --timeout 10m \
-f configs/kubernetes/active-active-values.yaml \
--set-string image.tag=${PLAYSBC_VERSION} \
--set-string rtpengine.image.tag=${PLAYSBC_VERSION}
```

If kubectl reports connection refused on 127.0.0.1, start Docker Desktop and the existing playsbc-control-plane container. Recreate the cluster only when `kind get clusters` no longer lists playsbc.

Azure AKS Administration



Required Azure Resources

- AKS resource group and cluster.
- Network resource group with static SIP and RTP public IP resources.
- Azure Container Registry containing all four v2.6.0 images.
- Network Contributor permission for the AKS identity on the network resource group.

Verify Azure Services

```
kubectl -n playsbc get pods -o wide
kubectl -n playsbc get svc \
playsbc-playsbc-azure-sip-public \
playsbc-playsbc-azure-rtp-public -o wide

kubectl -n playsbc describe svc \
playsbc-playsbc-azure-sip-public | sed -n '/Events:$/, $p'
```

A pending external IP is not solved by restarting PlaySBC. Check Service events, static public-IP names, resource groups, and AKS managed identity permission. Wait for both ingress addresses before registration, calls, or AKS regression.

Real-Device Administration

Validated Identities

Endpoint	User	Password	Transport
OBi1022	1001	secret-password	UDP
Zoiper	1002	secret-password	UDP

AKS Settings

- Proxy and registrar: Azure SIP public IP.
- SIP port: 5062 for UDP/TCP; 5061 for TLS where configured.
- Outbound proxy: blank for the validated baseline.
- Media: plain RTP/AVP with G.711; RTPengine advertises the Azure RTP public IP.

Local Settings

Use the Mac LAN IPv4 address and the isolated `kind-playsbc-real-device` context. The values profile uses fixed host ports and a Recreate rollout strategy.

Call Acceptance

- Both users register after the digest challenge.
- 1001 -> 1002 and 1002 -> 1001 ring, answer, and tear down cleanly.
- Both parties hear audio in both directions.
- RTPengine evidence reports caller-to-callee and callee-to-caller traffic.

Helm Configuration and Lifecycle

Group	Administrative Responsibility
image / rtpengine.image	Repository, immutable tag, and pull policy.
playsbc.config	SIP ports, advertised addresses, RTP range, media backend, TLS, and policy.
rtpengine	Enablement, control URL, RTP range, host networking, and advertised media IP.
cloud.azure	Static public-IP names, network resource group, and LB exposure.
observability	Prometheus/Grafana enablement, persistence, and retention.
authSecret	Lab digest users; production secret management is not provided.

```
helm -n playsbc list
helm -n playsbc get values playsbc -a
helm -n playsbc history playsbc
helm -n playsbc rollback playsbc <REVISION> --wait --timeout 10m

kubectl -n playsbc rollout status deployment/playsbc-playsbc --timeout=240s
kubectl -n playsbc rollout status deployment/playsbc-playsbc-rtpengine --timeout=240s
```

Verify configured images and the runtime-reported PlaySBC version after every upgrade.

Operations and Observability

Image and Version Verification

```
kubectl -n playsbc get deployment,statefulset \
-l 'app.kubernetes.io/instance=playsbc,app.kubernetes.io/name in (playsbc,playsbc-rtpengine)' \
-o custom-columns='KIND:.kind,NAME:.metadata.name,IMAGES:.spec.template.spec.containers[*].image'

kubectl -n playsbc exec deployment/playsbc-playsbc -- \
python3 -c 'import mini_call_server as s; print(s.PLAYSBC_VERSION)'
```

Health and Metrics

```
kubectl -n playsbc exec deployment/playsbc-playsbc -- \
python3 -c "import urllib.request;
print(urllib.request.urlopen('http://127.0.0.1:8080/readyz').read().decode())"

kubectl -n playsbc port-forward deployment/playsbc-playsbc-grafana 3000:3000
kubectl -n playsbc port-forward deployment/playsbc-playsbc-prometheus 9090:9090
```

Grafana visualizes Prometheus data. When a panel looks wrong, inspect the Prometheus query and raw PlaySBC metrics before changing dashboard colors or thresholds.

Regression and Evidence Administration

The canonical local Kubernetes suite contains 70 profiles. The launcher prints live progress and inhibits macOS sleep for the lifetime of a non-dry-run Job.

```
PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \  
python3 tools/run_k8s_regression_job.py \  
--all-profiles \  
--runner-image ghcr.io/sudheerkumarvatrapu/playsbc-k8s-regression:2.6.0 \  
--sipp-image ghcr.io/sudheerkumarvatrapu/playsbc-sipp:2.6.0 \  
--playsbc-image ghcr.io/sudheerkumarvatrapu/playsbc:2.6.0 \  
--rtengine-image ghcr.io/sudheerkumarvatrapu/playsbc-rtengine:2.6.0 \  
--set-playsbc-image --set-rtengine-image \  
--no-load-playsbc-image --no-load-rtengine-image --no-load-sipp-image \  
--kind-cluster playsbc
```

Evidence Contract

Artifact	Purpose
latest.html	Profile verdicts and links to retained evidence.
sipmsg.log	Combined readable SIP messages.
capture.pcap	Combined non-load signalling/media/network packet evidence.
pcap-legs.json	Expected roles, packet counts, SIP events, and B2BUA INVITE legs.
log.sip / log.media	Protocol and media decisions.
archive	Portable evidence bundle for review outside the cluster.

Security, HA, and Recovery

Security Baseline

- Use TLS secrets and trusted certificates for TLS labs.
- Keep credentials outside source-controlled values in any non-lab environment.
- Restrict public SIP, health, and RTP exposure with firewall and network policy controls.
- Use immutable image tags and verify digests before promotion.
- Review logs, transcripts, and media before sharing evidence.

HA Baseline

The public chart and suite include active-active, node-drain, shared-state, and failure-injection foundations. They are lab evidence, not a claim of lossless carrier-grade failover or RTPengine media-session migration.

Recovery Sequence

- Confirm kube context, node readiness, and current Helm revision.
- Inspect pod state and events before restarting workloads.
- Verify images, ConfigMap values, advertised addresses, and RTP range.
- Roll back Helm when a known-good revision is safer than an in-place repair.
- Collect logs and evidence before deleting a failed pod when possible.

Troubleshooting Guide

Symptom	Likely Cause	Action
127.0.0.1 refused	Docker/kind API stopped	Start Docker and control-plane; verify context.
InvalidImageName	Empty ACR/version variable	Export variables and verify the tag before Helm.
ImagePullBackOff	Missing tag or ACR auth	Check ACR tag and AKS pull role.
Azure IP pending	Static IP or identity permission	Describe Service; verify IP names and Network Contributor.
REGISTER absent	Wrong IP/port/transport	Test OPTIONS; inspect LB and endpoint profile.
One-way/no audio	Wrong RTP IP/range or NAT	Compare SDP, RTP LB, RTPengine interface, and PCAP.
Long-run time gap	Host sleep or shell disconnect	Use caffeine-enabled launcher and preserve remote Job.
PCAP lacks a leg	Missing/empty role capture	Inspect pcap-legs.json; v2.6.0 fails the evidence gate.

Capacity and Product Boundaries

What v2.6.0 Can Claim

- Repeatable lab registration, signalling, media, AI-foundation, HA-foundation, observability, and evidence scenarios.
- Validated OBi1022/Zoiper calls in documented AKS and local-LAN topologies.
- A structured path for measuring behavior through Docker, Kubernetes, AKS, and real devices.

What It Does Not Claim

- Production certification for 300,000 registered devices or 2,500 concurrent calls.
- Lossless mid-call migration for PlaySBC or RTPengine failures.
- Complete RFC 3261, carrier interconnect, STIR/SHAKEN, lawful intercept, emergency calling, or regulatory certification.
- Production distributed state, multi-zone recovery, or commercial support SLA.

Capacity statements must come from repeatable tests that publish topology, image digests, workload, duration, CPU, memory, packet rate, CPS, registrations per second, media quality, and failure behavior.

Command Reference

Inspect Workloads

```
kubectl -n playsbc get pods,svc,deploy,statefulset -o wide
```

Restart Observability

```
kubectl -n playsbc rollout restart deployment/playsbc-playsbc-grafana
kubectl -n playsbc rollout restart deployment/playsbc-playsbc-prometheus
kubectl -n playsbc rollout status deployment/playsbc-playsbc-grafana --timeout=180s
kubectl -n playsbc rollout status deployment/playsbc-playsbc-prometheus --timeout=180s
```

Monitor Calls and Media

```
kubectl -n playsbc logs -f \
-l app.kubernetes.io/instance=playsbc \
--all-containers=true --prefix --max-log-requests=10 --since=10m \
| grep -aE 'SIP (INVITE|ACK|BYE|CANCEL)|SIP TX response|SIP response|SDP SUMMARY|RTPENGINE|RTP
packet|RTCP|1001|1002'
```

Release Artifact Verification

```
shasum -a 256 -c release/helm/playsbc-2.6.0.tgz.sha256
helm lint charts/playsbc
helm template playsbc charts/playsbc \
-f configs/kubernetes/active-active-values.yaml >/tmp/playsbc.yaml
```

Canonical References

- docs/KUBERNETES_HELM_RUNBOOK.md - local Kubernetes administration.
- docs/AZURE_AKS.md - Azure creation, deployment, regression, and cleanup.
- docs/REAL_DEVICE_LAB.md - OBi1022 and Zoiper configuration and evidence.
- docs/OBSERVABILITY.md - Grafana and Prometheus operations.
- docs/AI_VOICE_GATEWAY.md - public AI lab baseline.
- release/RELEASE_NOTES_2.6.0.md - immutable release record.

Administrative Appendix A: Prerequisites

Common Tools

```
git --version
python3 --version
kubectl version --client
helm version --short
```

Local kind and Docker workflows also require Docker Desktop and kind. Azure Cloud Shell supplies `az`, `kubectl`, `helm`, and `Git`.

Repository and Version

```
export PLAYSBC_VERSION=2.6.0

git clone --branch "v$PLAYSBC_VERSION" --depth 1 \
https://github.com/sudheerkumarvatrapu/PlaySBC.git \
"PlaySBC-v$PLAYSBC_VERSION"

cd "PlaySBC-v$PLAYSBC_VERSION"
```

Administrative Appendix B: Cluster Creation

kind

```
open -a Docker
until docker info >/dev/null 2>&1; do sleep 5; done

kind create cluster --name playsbc --wait 180s
kubectl config use-context kind-playsbc
kubectl create namespace playsbc --dry-run=client -o yaml | kubectl apply -f -
kubectl config set-context --current --namespace=playsbc
```

Minikube Compatibility Lane

```
minikube start --driver=docker --profile playsbc
kubectl config use-context playsbc
kubectl create namespace playsbc --dry-run=client -o yaml | kubectl apply -f -
kubectl config set-context --current --namespace=playsbc
```

Minikube driver behavior varies. The Docker driver requires Docker Desktop; a VM driver requires its matching hypervisor. kind remains the canonical full-regression lane.

Administrative Appendix C: Azure AKS Provisioning

Variables and Providers

```
export LOCATION=eastus
export AKS_RG=playsbc-aks-rg
export NETWORK_RG=playsbc-network-rg
export AKS_NAME=playsbc-aks
export ACR_NAME=playsbcacr$RANDOM
export SIP_PIP_NAME=playsbc-sip-pip
export RTP_PIP_NAME=playsbc-rtp-pip

for PROVIDER in Microsoft.ContainerService Microsoft.ContainerRegistry \
Microsoft.Network Microsoft.Compute Microsoft.ManagedIdentity; do
az provider register --namespace "$PROVIDER"
done

az group create -n "$AKS_RG" -l "$LOCATION"
az group create -n "$NETWORK_RG" -l "$LOCATION"
```

ACR and AKS

```
az acr create -g "$AKS_RG" -n "$ACR_NAME" --sku Basic

az aks create \
-g "$AKS_RG" -n "$AKS_NAME" -l "$LOCATION" \
--tier free --node-count 1 --node-vm-size Standard_D2as_v7 \
--load-balancer-sku standard --attach-acr "$ACR_NAME" \
--generate-ssh-keys

az aks get-credentials -g "$AKS_RG" -n "$AKS_NAME" --overwrite-existing
kubectl get nodes
```

Static Public IPs and AKS Identity

```
az network public-ip create \
-g "$NETWORK_RG" -n "$SIP_PIP_NAME" -l "$LOCATION" \
--sku Standard --allocation-method Static

az network public-ip create \
-g "$NETWORK_RG" -n "$RTP_PIP_NAME" -l "$LOCATION" \
--sku Standard --allocation-method Static

AKS_OBJECT_ID=$(az aks show -g "$AKS_RG" -n "$AKS_NAME" \
--query identity.principalId -o tsv)
NETWORK_RG_ID=$(az group show -n "$NETWORK_RG" --query id -o tsv)

az role assignment create \
--assignee-object-id "$AKS_OBJECT_ID" \
--assignee-principal-type ServicePrincipal \
--role "Network Contributor" \
--scope "$NETWORK_RG_ID"
```

Import v2.6.0 Images

```
export PLAYSBC_VERSION=2.6.0

for IMAGE in playsbc playsbc-rtpengine playsbc-k8s-regression playsbc-sipp; do
az acr import --name "$ACR_NAME" \
--source "ghcr.io/sudheerkumarvatrapu/$IMAGE:$PLAYSBC_VERSION" \
--image "$IMAGE:$PLAYSBC_VERSION" --force
done
```

Install the AKS Baseline

Use `configs/kubernetes/aks-values.yaml` as the base and set the current ACR, static IP resource names, network resource group, and advertised addresses. Wait for both SIP and RTP Services to receive external IPs before calls or regression.

```
export ACR_LOGIN_SERVER=$(az acr show -g "$AKS_RG" -n "$ACR_NAME" \
--query loginServer -o tsv)
export NODE_RG=$(az aks show -g "$AKS_RG" -n "$AKS_NAME" \
--query nodeResourceGroup -o tsv)
```

```
export SIP_PUBLIC_IP=$(az network public-ip show -g "$NETWORK_RG" \
-n "$SIP_PIP_NAME" --query ipAddress -o tsv)
export RTP_PUBLIC_IP=$(az network public-ip show -g "$NETWORK_RG" \
-n "$RTP_PIP_NAME" --query ipAddress -o tsv)

helm upgrade --install playsbc \
"https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v${PLAYSBC_VERSION}/playsbc-${PLAYSBC_VERSION}
.tgz" \
-n playsbc --create-namespace --atomic --wait --timeout 10m \
-f configs/kubernetes/aks-values.yaml \
--set cloud.azure.nodeResourceGroup="$NODE_RG" \
--set cloud.azure.sip.public.publicIPResourceGroup="$NETWORK_RG" \
--set cloud.azure.sip.public.publicIPName="$SIP_PIP_NAME" \
--set cloud.azure.media.public.publicIPResourceGroup="$NETWORK_RG" \
--set cloud.azure.media.public.publicIPName="$RTP_PIP_NAME" \
--set image.repository="$ACR_LOGIN_SERVER/playsbc" \
--set-string image.tag="$PLAYSBC_VERSION" \
--set rtpengine.image.repository="$ACR_LOGIN_SERVER/playsbc-rtpengine" \
--set-string rtpengine.image.tag="$PLAYSBC_VERSION" \
--set-string rtpengine.advertisedIP="$RTP_PUBLIC_IP" \
--set-string playsbc.config.sip_advertised_ip="$SIP_PUBLIC_IP" \
--set-string playsbc.config.b2bua_advertised_ip="$SIP_PUBLIC_IP"
```

Administrative Appendix D: AKS Regression

AKS regression uses 12 cloud-readiness profiles and one PlaySBC plus one RTPengine by default to control lab cost.

```
PYTHONPYCACHEPREFIX=/tmp/playsbc-pycache \  
python3 tools/run_k8s_regression_job.py \  
--aks-profiles \  
--runner-image "$ACR_LOGIN_SERVER/playsbc-k8s-regression:$PLAYSBC_VERSION" \  
--runner-image-pull-policy Always \  
--sipp-image "$ACR_LOGIN_SERVER/playsbc-sipp:$PLAYSBC_VERSION" \  
--sipp-image-pull-policy Always \  
--playsbc-image "$ACR_LOGIN_SERVER/playsbc:$PLAYSBC_VERSION" \  
--rtpengine-image "$ACR_LOGIN_SERVER/playsbc-rtpengine:$PLAYSBC_VERSION" \  
--set-playsbc-image --set-rtpengine-image \  
--aks-load-balancer-wait-timeout 1200 \  
--job-timeout 3600
```

Download `latest-aks-regression.tgz` from Cloud Shell immediately after the run because the session may be ephemeral.

Administrative Appendix E: Local Real-Device Cluster

```
export REAL_DEVICE_CLUSTER=playsbc-real-device
export REAL_DEVICE_CONTEXT=kind-playsbc-real-device
export LAN_IF=$(route -n get default | awk '/interface:/{print $2; exit}')
export LAN_IP=$(ipconfig getifaddr "$LAN_IF")
: "${LAN_IP:?Could not determine Mac LAN IPv4}"

kind create cluster \
--name "$REAL_DEVICE_CLUSTER" \
--config configs/kubernetes/kind-real-device-cluster.yaml \
--wait 180s

helm upgrade --install playsbc \
"https://github.com/sudheerkumarvatrapu/PlaySBC/releases/download/v${PLAYSBC_VERSION}/playsbc-${PLAYSBC_VERSION}.tgz" \
--kube-context "$REAL_DEVICE_CONTEXT" \
-n playsbc --create-namespace --atomic --wait --timeout 5m \
-f configs/kubernetes/kind-real-device-values.yaml \
--set-string localRealDevice.lanIPv4="$LAN_IP" \
--set-string playsbc.config.sip_advertised_ip="$LAN_IP" \
--set-string playsbc.config.b2bua_advertised_ip="$LAN_IP" \
--set-string rtpengine.advertisedIP="$LAN_IP" \
--set-string image.tag="$PLAYSBC_VERSION" \
--set-string rtpengine.image.tag="$PLAYSBC_VERSION"
```

The TLS lane also requires the `playsbc-real-device-tls` Secret described by the chart values. UDP registration and calls can be validated before TLS device trust is configured.

Capture Real-Device Evidence

```
PYTHONPYCACHEPREFIX=/private/tmp/playsbc-pycache \
python3 tools/run_real_device_capture.py \
--context "$REAL_DEVICE_CONTEXT" \
--namespace playsbc \
--duration 120 \
--capture-image nicolaka/netshoot:latest
```

Administrative Appendix F: Cleanup

Local Clusters

```
helm --kube-context kind-playsbc-real-device -n playsbc uninstall playsbc
kind delete cluster --name playsbc-real-device

helm --kube-context kind-playsbc -n playsbc uninstall playsbc
kind delete cluster --name playsbc
```

Azure

Set the active subscription from the current account; do not paste an old subscription ID from another tenant.

```
export SUB_ID=$(az account show --query id -o tsv)
export AKS_RG=playsbc-aks-rg
export NETWORK_RG=playsbc-network-rg

for RG in "$AKS_RG" "$NETWORK_RG"; do
  if [ "$(az group exists --subscription "$SUB_ID" --name "$RG")" = true ]; then
    az group delete --subscription "$SUB_ID" --name "$RG" --yes --no-wait
  fi
done
```

Resource-group deletion is asynchronous. A stale kubectl context may continue to show cached objects briefly and later fail DNS/API access after the AKS control plane is removed.